ParkLink: una plataforma basada en microservicios para la reserva de estacionamiento urbano y el uso colaborativo de espacios

Thiago Fernando Baez Santiago Domínguez Facundo Luzzi
Santiago Jorge Varga
Facultad de Ingeniería, Universidad de Buenos Aires (UBA), Buenos Aires, Argentina
{tbaez, sdominguez, fluzzi, svarga}@fi.uba.ar

25 de octubre de 2026

Resumen

Encontrar un lugar para estacionar es un problema persistente y costoso en las ciudades densamente pobladas. En Buenos Aires, los conductores dedican en promedio 15 minutos por viaje a buscar un espacio, con pérdidas estimadas de $5.612 ARS por búsqueda en tiempo, combustible, multas y precios desfavorables. Al mismo tiempo, una gran proporción de cocheras privadas permanece ociosa por la ausencia de un canal digital confiable que conecte oferta y demanda. Este trabajo presenta ParkLink, un mercado digital bilateral que permite a los conductores buscar, reservar y pagar el estacionamiento antes de salir, y a los propietarios publicar y administrar sus espacios. El sistema se construye sobre una arquitectura de microservicios independientes (Python/FastAPI, Go, NestJS/TypeScript, Node.js/Express), una aplicación móvil multiplataforma en React Native y una base de datos espacial PostgreSQL/PostGIS. La comunicación asíncrona entre servicios se gestiona mediante RabbitMQ y el procesamiento de pagos se integra con Mercado Pago mediante notificaciones verificadas criptográficamente. Una metodología híbrida Scrum–Kanban organizó trece sprints de dos semanas a lo largo de un ciclo de desarrollo de seis meses. La validación incluyó 128 archivos de pruebas automatizadas ejecutados en una canalización de integración continua, pruebas de extremo a extremo en dispositivos físicos, una encuesta de mercado a 180 participantes que produjo una intención de uso habitual del 70 % y una entrevista con un referente de la operación comercial de estacionamientos. El MVP resultante demuestra la viabilidad técnica y comercial de transformar la búsqueda de estacionamiento de un problema secuencial de detención óptima en una elección simultánea e informada, reduciendo la circulación innecesaria y favoreciendo un uso más eficiente de la infraestructura urbana existente.

Palabras clave: estacionamiento urbano; microservicios; mercado bilateral; aplicación móvil; búsqueda geoespacial; economía colaborativa

1. Introducción y contexto

La movilidad urbana es uno de los desafíos centrales de las ciudades contemporáneas, y el estacionamiento ocupa un lugar particular: constituye el tramo final de casi todo viaje en automóvil y, simultáneamente, el segmento que ha recibido menor atención tecnológica. En Buenos Aires, esta brecha es especialmente visible. La ciudad registra aproximadamente 1,2 millones de ingresos vehiculares diarios [11], pero su oferta de estacionamiento continúa fragmentada en tres canales desconectados: zonas públicas reguladas con parquímetro, playas comerciales sin reserva anticipada y portales de avisos clasificados enfocados casi exclusivamente en alquileres mensuales.

Desde la perspectiva de la teoría de decisiones, un conductor que busca estacionamiento enfrenta un **problema clásico de detención óptima** [1]. La formulación canónica —conocida como el problema de estacionamiento de MacQueen y Miller [2]— modela a un conductor que avanza hacia un destino, solo puede observar el espacio que tiene inmediatamente delante, no puede regresar a una opción vista previamente y debe decidir en cada momento si toma el espacio actual o continúa. Bajo la estrategia de umbral óptimo, incluso en el mejor caso la probabilidad de elegir la mejor opción disponible es de apenas 37 % [3]. La causa subyacente no es la escasez de espacios, sino la escasez de información: la oferta existe, pero el conductor no puede verla, compararla ni comprometerse con ella antes de iniciar el viaje.

En paralelo a esta ineficiencia del lado de la demanda, persiste otra del lado de la oferta: una gran proporción de cocheras residenciales y playas comerciales permanece subutilizada durante buena parte del día, sin un canal accesible para que sus propietarios las ofrezcan por períodos cortos. El costo económico de esta doble ineficiencia puede cuantificarse. Nuestro análisis de datos de Buenos Aires correspondientes a 2026 estima **$5.612 ARS por búsqueda** —compuestos por tiempo perdido ($3.464 ARS), combustible ($556 ARS), riesgo estadístico de multa ($792 ARS) y exposición a precios desfavorables ($800 ARS)— en un contexto de 180.000–270.000 búsquedas activas de estacionamiento por día en la ciudad.

ParkLink fue diseñada para cerrar esta brecha transformando la búsqueda de estacionamiento, de un proceso secuencial y ciego, en una elección simultánea y completamente informada. La plataforma permite a los conductores visualizar en un mapa georreferenciado todos los espacios disponibles cerca de su destino, consultar disponibilidad y precios en tiempo real, realizar reservas anticipadas por hora o por estadías prolongadas y pagar digitalmente antes de salir. Los propietarios publican sus espacios, definen capacidad, horarios y precios, aprueban solicitudes entrantes y reciben el 100 % de la tarifa publicada. La plataforma cobra al conductor una comisión de servicio del 8 % por cada reserva completada, financiando la operación sin imponer costos del lado de la oferta.

El resto de este trabajo se organiza de la siguiente manera. La Sección 2 revisa las soluciones existentes y los trabajos relacionados. La Sección 3 describe la arquitectura y el diseño técnico del sistema. La Sección 4 presenta la metodología de desarrollo. La Sección 5 informa los resultados de validación y los hallazgos principales. La Sección 6 expone las conclusiones y las líneas de trabajo futuro.

2. Trabajos relacionados

2.1. Sistemas de información y navegación para estacionamiento

Las plataformas de navegación como Google Maps [9] y Waze [8] incorporan disponibilidad estimada de estacionamiento a partir de patrones agregados, pero ninguna ofrece reservas confirmadas, inventario en tiempo real de propietarios privados ni pagos integrados. Su información de estacionamiento es indicativa, no transaccional.

2.2. Gestión del estacionamiento medido

Blinkay [5] es la aplicación dominante de estacionamiento medido en Argentina y permite pagar mediante el teléfono las zonas reguladas en la vía pública de Buenos Aires y otras ciudades. Su arquitectura está estrechamente vinculada a contratos municipales, cubre únicamente espacios públicos en la vía pública y no ofrece ningún mecanismo de reserva. El conductor todavía debe llegar y esperar encontrar disponibilidad.

2.3. Plataformas internacionales de reservas

SpotHero y ParkWhiz [6] representan el estado del arte de la reserva anticipada de estacionamiento en Estados Unidos, al ofrecer inventario en tiempo real, comparación de precios y confirmación instantánea de reservas en cocheras comerciales. EasyPark [7] cubre un alcance similar en más de 20 países europeos, combinando el pago de zonas municipales con la reserva en algunas cocheras privadas. Ambas validan el modelo técnico y comercial; sin embargo, ninguna opera en Argentina o Latinoamérica ni aborda la dimensión de economía colaborativa de las cocheras residenciales privadas.

2.4. Portales de avisos clasificados

Portales como MercadoLibre [10] son el principal canal mediante el cual los propietarios privados de Argentina publican espacios de estacionamiento, aunque exclusivamente para alquileres mensuales de plazo fijo. No existe disponibilidad en tiempo real, pago transaccional dentro de la plataforma ni coordinación automática entre conductor y propietario.

2.5. Síntesis y brecha de investigación

La Tabla 1 resume la cobertura de las soluciones existentes frente a los cinco criterios que guiaron el diseño de ParkLink: presencia en el mercado local, reserva anticipada, disponibilidad confirmada en tiempo real, pago integrado y acceso a propietarios privados.

Cuadro 1: Comparación de las soluciones de estacionamiento existentes según criterios clave.

Solución	Local	Reserva	Disp. real	Pago	Privada
Blinkay	✓	×	parcial	✓	×
SpotHero/ParkWhiz	×	✓	✓	✓	×
EasyPark	×	parcial	parcial	✓	parcial
Google Maps/Waze	✓	×	estimada	×	×
Avisos clasificados	✓	×	×	×	✓
ParkLink	✓	✓	✓	✓	✓

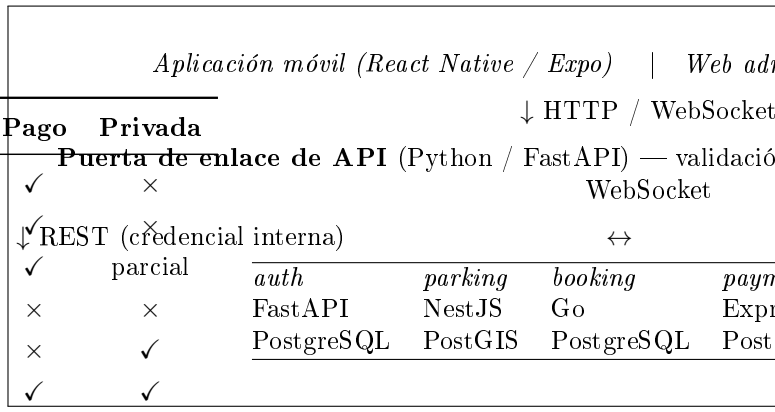


Figura 1: Arquitectura de ParkLink: cinco microservicios independientes detrás de una única puerta de enlace; RabbitMQ desacopla los eventos asíncronos.

Ningún actor existente del mercado argentino ofrece simultáneamente búsqueda geolocalizada, reserva anticipada, disponibilidad confirmada en tiempo real, pago integrado y acceso a espacios de propiedad privada. ParkLink fue construida precisamente para ocupar esta brecha.

3. Diseño y arquitectura del sistema

3.1. Descripción general de la arquitectura

ParkLink sigue una arquitectura de microservicios independientes en la que cada dominio de negocio se implementa como un servicio autónomo con su propia base de datos, repositorio y canalización de despliegue. Una única puerta de enlace de API constituye el punto de entrada público, centralizando la autenticación JWT, la limitación de frecuencia, las cabeceras de seguridad HTTP y las conexiones WebSocket en tiempo real. Este enfoque permite escalar de forma independiente, aislar fallas y utilizar tecnologías heterogéneas entre los servicios.

Figure 1 illustrates the component topology.

3.2. Servicios del servidor

Puerta de enlace de API (Python / FastAPI). El único punto de entrada perimetral valida los tokens de acceso, agrega una credencial interna confiable para los servicios posteriores, aplica límites de frecuencia por cliente y mantiene el canal WebSocket mediante el cual la aplicación móvil recibe notificaciones en tiempo real. Centralizar la autenticación en la puerta de enlace evita que cada servicio deba implementar su propia lógica perimetral.

Servicio de autenticación e identidad (Python / FastAPI). Administra el ciclo de vida completo de la cuenta: registro guiado con códigos de verificación de seis dígitos para correo electrónico y teléfono, inicio de sesión, renovación rotativa de tokens, gestión de sesiones por dispositivo, recuperación de contraseñas y eliminación de cuentas. Las contraseñas se almacenan mediante derivación de claves bcrypt y los tokens de acceso son JWT de corta duración. El servicio constituye la única fuente de verdad para la identidad de los usuarios y expone datos de los vehículos de los conductores, cuentas bancarias de los propietarios para las liquidaciones e información de perfil.

Servicio de espacios de estacionamiento y geolocalización (TypeScript / NestJS / PostGIS). Gestiona el ciclo de publicación de los espacios de estacionamiento: coordenadas espaciales, fotografías, precios por hora y por estadía, horarios semanales, fechas bloqueadas, capacidad, documentos de verificación y estado de moderación. Las búsquedas por proximidad se resuelven mediante índices espaciales en una base de datos PostgreSQL

con la extensión PostGIS, que permite consultas eficientes por radio y geocodificación de direcciones. Este servicio es propietario de la configuración que consulta el servicio de reservas para validar horarios y calcular precios de estadía.

Motor de reservas y disponibilidad (Go). Es el núcleo transaccional de la plataforma. La disponibilidad se calcula como la diferencia de conjuntos entre los intervalos del horario publicado de un espacio y sus reservas activas —un problema estructurado de aritmética de intervalos—. La seguridad ante concurrencia se garantiza mediante bloqueos de filas a nivel de base de datos dentro de la transacción de reserva, combinados con una retención temporal del espacio mientras el conductor completa el pago, lo que evita reservas duplicadas ante solicitudes simultáneas. El servicio también gestiona la aprobación de reservas, el ingreso y egreso presenciales mediante códigos PIN, extensiones, cancelaciones, cargos pendientes, reseñas de usuarios, mensajería por reserva, solicitudes de soporte y liquidaciones de ganancias a propietarios. Se eligió Go para este servicio por su modelo nativo de concurrencia y su rendimiento predecible bajo carga.

Servicio de pasarela de pagos (Node.js / Express). Se integra con Mercado Pago: genera preferencias de pago, recibe notificaciones webhook y verifica su firma criptográfica HMAC, confirma la reserva asociada y procesa reembolsos con reintentos automáticos ante fallas transitorias. Aislar la lógica de pagos en su propio servicio garantiza que la latencia o las fallas de la pasarela nunca afecten la búsqueda de espacios ni otras funciones de la plataforma. Cada operación es idempotente para gestionar entregas duplicadas de webhooks.

Servicio de notificaciones (Python / FastAPI / RabbitMQ). Consume eventos de dominio publicados por los servicios de reservas y pagos en un intermediario RabbitMQ. Para cada evento, registra la notificación en la bandeja de entrada del usuario, envía una notificación mediante Firebase Cloud Messaging y señala al WebSocket de la puerta de enlace que actualice inmediatamente la aplicación abierta. Los eventos duplicados se detectan y descartan para evitar notificaciones repetidas. Como este servicio no expone una interfaz pública, su desarrollo puede avanzar de manera independiente del resto.

3.3. Aplicaciones de interfaz

Aplicación móvil (React Native / Expo). Una aplicación multiplataforma para Android e iOS que ofrece experiencias de navegación diferenciadas: el flujo del conductor, centrado en búsqueda, reserva y pago; y el flujo del propietario, centrado en la gestión de espacios, aprobación de solicitudes e informes financieros. El mapa utiliza Google Maps Platform con geocodificación de Google Places. La aplicación admite varios idiomas y modo oscuro.

Panel web administrativo (React Native Web / Expo). Una aplicación web interna mediante la cual el personal de ParkLink busca usuarios, modera publicaciones y reseñas de espacios, gestiona liquidaciones y reembolsos, atiende solicitudes de soporte, consulta métricas de uso y configura la comisión de servicio de la plataforma.

3.4. Decisiones clave de diseño técnico

3.4.1. Diseño de contratos API.

Como los servicios de interfaz y del servidor se desarrollaron en paralelo, el contrato de API de cada servicio se acordó y fijó antes de comenzar la implementación. La capa de acceso a datos de la aplicación móvil se diseñó con las mismas firmas de métodos que los puntos de acceso reales, de modo que reemplazar datos simulados por llamadas al servidor en vivo no requiriera cambios en las pantallas.

3.4.2. Persistencia políglota.

Cada servicio es propietario de su propio esquema PostgreSQL; ningún servicio lee directamente la base de datos de otro. Las necesidades de datos entre servicios se resuelven mediante llamadas REST síncronas a través de la puerta de enlace o mediante el consumo de eventos de dominio. Este límite refuerza el aislamiento de dominios y permite la evolución independiente de los esquemas.

3.4.3. Asincronía orientada a eventos.

Los procesos secundarios —notificaciones, disparadores de liquidaciones y enlaces de analítica— se desacoplan de las transacciones principales mediante RabbitMQ. La garantía de entrega al menos una vez, combinada con el procesamiento idempotente de eventos, asegura un comportamiento correcto ante reintentos sin bloquear el camino crítico de las reservas.

3.4.4. Infraestructura y observabilidad.

Todos los componentes se empaquetan en contenedores Docker y se orquestan mediante un repositorio raíz que utiliza submódulos Git para componer todos los servicios. El sistema completo puede iniciarse localmente con un único comando. Sentry está integrado en los servicios para el seguimiento de errores y Cloudflare R2 almacena las fotografías de los espacios.

4. Metodología de desarrollo

4.1. Marco ágil híbrido

El desarrollo se gestionó mediante una metodología híbrida que combinó prácticas de Scrum y Kanban, elegida para adaptarse a la construcción iterativa de un sistema distribuido de ocho servicios cuyos requisitos evolucionaron a medida que maduró el producto. La metodología no se aplicó de manera uniforme, sino que se adaptó a la naturaleza específica de cada fase del proyecto.

4.2. Fases del proyecto y organización de los sprints

El proyecto se desarrolló de abril a octubre de 2026, durante trece sprints de dos semanas agrupados en tres fases.

Fase 1 — Propuesta y factibilidad (sprints 1–5). Se enfocó en definir qué construir y validar su viabilidad. Los entregables fueron artefactos de investigación y análisis: estudio de mercado, análisis competitivo, alcance del MVP, arquitectura preliminar e identidad de marca. La fase concluyó con la presentación de la propuesta de proyecto y la firma del acuerdo con los tutores.

Fase 2 — Desarrollo del MVP bajo Scrum (sprints 6–12). Se concentró en la construcción del producto. Cada sprint concluyó con un incremento funcional de software. Primero se construyó la aplicación móvil con una capa completa de datos simulados y luego se conectó progresivamente con los servicios reales. Cada integrante trabajó contra contratos simulados para los servicios aún no disponibles, evitando bloqueos causados por dependencias. Cada revisión de sprint se realizó ante los tutores del proyecto, quienes validaron el incremento y proporcionaron retroalimentación.

Fase 3 — Validación y cierre (sprint 13 y cierre progresivo). Con el MVP integrado, esta fase abordó el cumplimiento legal, la validación de mercado y la incorporación de los comentarios de los usuarios al producto. Después del sprint 13, las iteraciones de duración fija se reemplazaron por un flujo Kanban para corregir errores, preparar la documentación y elaborar el informe final.

4.3. Organización del equipo y herramientas

El equipo de cuatro integrantes asignó un responsable principal a cada servicio, lo que permitió el desarrollo paralelo. La Tabla 2 resume las responsabilidades.

Cuadro 2: Responsabilidades de los componentes por integrante del equipo.

Integrante	Responsabilidades principales
Thiago F. Baez	Interfaz (móvil + web administrativa), de usuarios
Santiago Domínguez	Responsable de calidad, servicio de pag
Facundo Luzzi	Motor de reservas, servicio de notificaci
Santiago J. Varga	Servicio de estacionamiento, calidad, do

El código fuente se organizó en una organización de GitHub con un repositorio por servicio, un repositorio independiente para la aplicación móvil y un repositorio raíz que los agregaba mediante submódulos Git. Las solicitudes de incorporación de cambios activaban canalizaciones de GitHub Actions que ejecutaban analizadores de estilo, análisis estático, verificaciones de tipos y pruebas automatizadas. Cada repositorio mantenía su propia documentación de contratos, con la descripción de puntos de acceso, modelos de datos y decisiones de diseño, que servía como especificación de interfaz compartida por el equipo.

La planificación de los sprints utilizó Planning Poker con estimación de puntos de historia de Fibonacci, priorizando la complejidad por sobre el tiempo para permitir la comparación entre tareas diferentes. Las retrospectivas posteriores a cada sprint ajustaron las prácticas de trabajo a partir de la reflexión del equipo.

5. Resultados y validación

La validación se realizó en dos planos complementarios: corrección técnica y adecuación al mercado.

5.1. Pruebas automatizadas e integración continua

La estrategia de pruebas siguió el modelo de pirámide de pruebas: una base amplia de pruebas unitarias que verifican reglas de negocio aisladas, una capa intermedia de pruebas de integración que verifica cada servicio contra su base de datos y sus dependencias, y un vértice estrecho de pruebas de extremo a extremo que ejercita flujos completos de usuario. La Tabla 3 muestra la cantidad de archivos de pruebas automatizadas por componente al cierre del desarrollo.

Cuadro 3: Archivos de pruebas automatizadas por componente.

Componente	Marco	Archivos de pruebas
booking-service	Go testing, testify, gomock	72
notifications-service	pytest	17
auth-service	pytest	10
mobile	Jest	9
parking-service	Jest	7
payment-service	Jest, supertest	7
gateway	pytest	6
Total		**128**

La distribución refleja la concentración del riesgo: el servicio de reservas, que encapsula la mayoría de las reglas de negocio, representa el 56 % de todos los archivos de pruebas. Entre los escenarios críticos verificados automáticamente se encuentran:

- **Cálculo de disponibilidad**: Resta de reservas activas a los horarios publicados, incluida la conversión de zonas horarias y los casos límite de los horarios.
- **Seguridad de reservas concurrentes**: Dos solicitudes simultáneas para el mismo espacio no pueden producir una reserva duplicada; la retención temporal vence correctamente si el pago no se completa dentro del intervalo.
- **Códigos de ingreso y egreso**: Generación, validación y cálculo correctos del cargo por permanencia excedida.
- **Integridad de webhooks de pago**: Verificación de firma HMAC, confirmación idempotente de reservas y reintentos automáticos de reembolsos.
- **Deduplicación de notificaciones**: Un evento duplicado no debe generar dos notificaciones; los mensajes malformados no deben detener la cola.
- **Perímetro de la puerta de enlace**: Validación de tokens, rechazo de solicitudes no autenticadas y eliminación de cabeceras internas inyectadas por clientes externos.

Todas las pruebas se ejecutaron automáticamente en cada envío mediante canalizaciones de GitHub Actions que también ejecutaban analizadores de estilo, análisis estático y verificadores estrictos de tipos, asegurando que ningún error detectable llegara a la rama principal.

5.2. Pruebas de integración de extremo a extremo

Además de las pruebas por servicio, un script específico de extremo a extremo se ejecutó contra el sistema completamente integrado: creó una cuenta de propietario, publicó un espacio con horario y precio, creó una cuenta de conductor, consultó la disponibilidad, inició una reserva y verificó el resultado de cada paso. Este script proporcionó una confirmación rápida de ausencia de regresiones después de cada evento de integración. Se realizaron pruebas manuales en dispositivos físicos Android e iOS, validando la entrega de notificaciones, la ubicación GPS y el flujo de redirección y retorno de pagos de Mercado Pago —aspectos que los emuladores no reproducen fielmente—. Los pagos se ejercitaron en el entorno de pruebas de Mercado Pago con cuentas de compradores de prueba y sin movimiento de dinero real.

5.3. Encuesta de validación de mercado

A comienzos de septiembre de 2026, el equipo realizó una encuesta de validación de mercado distribuida mediante Instagram y dirigida a adultos de Buenos Aires con al menos experiencia ocasional de conducción. Se recopilaron **180 respuestas**, que abarcaron tanto a conductores como a propietarios.

Hallazgos principales:

- **Alta tasa de aceptación**: La posibilidad de salir de casa con un espacio de estacionamiento reservado y pagado recibió calificaciones de 4 o 5 sobre 5 por parte de la gran mayoría de los participantes.
- **Intención de uso habitual**: Aproximadamente el **70 %** de los participantes indicó que utilizaría la aplicación habitualmente, con frecuencias desde varias veces al mes hasta diariamente.

- **Principales motivadores**: Precios previsibles (incluida la posibilidad de comparar opciones antes de comprometerse), cercanía al destino y seguridad del vehículo (preferencia por espacios privados cerrados frente al estacionamiento en la calle).

- **Disposición a pagar**: El rango de precios que los conductores declararon estar dispuestos a pagar fue ligeramente inferior al promedio de mercado utilizado en nuestras proyecciones de ingresos, lo que refuerza el valor estratégico de incorporar propietarios privados capaces de ofrecer precios inferiores a los de las playas comerciales.

Los puntos de fricción identificados por la encuesta se incorporaron como decisiones de producto: la llegada con el automóvil del ocupante anterior todavía presente (abordada mediante códigos de egreso confirmados), la incertidumbre sobre cómo ingresar a un espacio privado (abordada mediante instrucciones de acceso en la aplicación vinculadas a cada reserva) y la duda sobre la existencia física del espacio publicado (abordada mediante verificación de identidad del propietario y moderación previa a la publicación).

5.4. Validación con un referente de la industria

En la última semana de septiembre de 2026, el equipo realizó una entrevista con Miguel Botbol, un referente de la operación comercial de estacionamientos en Buenos Aires, con aproximadamente 70 playas en la ciudad y sus alrededores. La reunión se realizó con el MVP completamente integrado, lo que permitió una demostración en vivo de la experiencia completa del conductor (buscar, reservar y pagar), la experiencia del propietario (publicar, administrar y recibir ganancias) y el panel administrativo.

Botbol validó el valor de la plataforma para los operadores comerciales, no solo para los propietarios privados, y confirmó que un canal digital unificado que integre toda la oferta —privada y comercial, pequeña y grande— con reserva anticipada y pago integrado representa una necesidad real aún no satisfecha en el mercado. También aportó conocimientos operativos que orientaron la hoja de ruta de trabajo futuro descrita en la Sección 6.

5.5. Validación legal

Antes de habilitar el uso para usuarios reales, se redactaron y publicaron los términos de servicio y la política de privacidad de la plataforma. Los términos de servicio regulan reservas, pagos, cancelaciones, liquidaciones a propietarios y responsabilidades; exigen que los usuarios sean mayores de edad y obligan a los propietarios a declarar su derecho a publicar el espacio. Se preservan explícitamente los derechos de los consumidores garantizados por la Ley argentina 24.240. La política de privacidad se redactó de conformidad con la Ley 25.326 de Protección de Datos Personales, detallando qué datos se recopilan, por qué, con quién se comparten, durante cuánto tiempo se conservan y cómo pueden los usuarios ejercer sus derechos de acceso, rectificación y supresión. La aceptación de ambos documentos es un paso obligatorio del flujo de registro.

5.6. Resultado consolidado

El proceso de validación concluyó con los siguientes resultados confirmados:

1. Un MVP funcional con todos los flujos críticos cubiertos por 128 archivos de pruebas automatizadas ejecutados en cada cambio de código, y el recorrido completo buscar–reservar–pagar–notificar verificado en dispositivos reales y redes distribuidas sin fallas de concurrencia.

2. Demanda de mercado confirmada en ambos lados de la plataforma, con una intención de uso habitual del 70 % y una identificación clara de los puntos de fricción resueltos en el producto antes del lanzamiento.

3. Aval de un experto del dominio que valida el valor para operadores comerciales y el potencial de transformación del estacionamiento urbano.

4. Cumplimiento legal integral de la normativa argentina de protección del consumidor y privacidad de datos.

6. Conclusiones y trabajo futuro

6.1. Conclusiones

Este trabajo presentó ParkLink, un mercado digital bilateral que aborda la ineficiencia persistente de la búsqueda de estacionamiento urbano en Buenos Aires. La contribución central es arquitectónica y conceptual: al combinar en una única plataforma la agregación georreferenciada de la oferta, la reserva anticipada, el pago integrado y el acceso colaborativo a espacios de propiedad privada, ParkLink transforma la búsqueda de estacionamiento de un

problema secuencial y ciego de detención óptima en una elección simultánea completamente informada.

Desde el punto de vista técnico, la plataforma demuestra que una arquitectura de microservicios independientes con una selección políglota de tecnologías, asincronía orientada a eventos y una estrategia de API basada en contratos no solo es viable, sino también prácticamente superior para un proyecto de este alcance y tamaño de equipo: permitió el desarrollo paralelo de ocho componentes, aisló fallas y habilitó entregas independientes sin dependencias bloqueantes.

La metodología híbrida Scrum–Kanban, organizada en trece sprints de dos semanas y acompañada por revisiones quincenales de los tutores, proporcionó estructura suficiente para mantener la previsibilidad durante un horizonte de seis meses y, al mismo tiempo, adaptarse a los requisitos cambiantes de un mercado bilateral. El esfuerzo acumulado ascendió aproximadamente a 1.600 horas de equipo.

La validación confirmó las cuatro hipótesis centrales: corrección técnica bajo carga concurrente, viabilidad comercial mediante una señal de intención de uso habitual del 70 %, alineación con la industria a través del aval de un experto y factibilidad legal bajo la normativa argentina vigente.

6.2. Trabajo futuro

Tres líneas estratégicas permiten extender ParkLink más allá del MVP actual:

Detección de ocupación mediante IoT y visión computacional. Reemplazar la disponibilidad basada en horarios declarados manualmente por la detección física de ocupación en tiempo real mediante visión computacional de aprendizaje profundo, reconocimiento automático de matrículas (LPR/ALPR) para un ingreso sin fricciones y sensores IoT ultrasónicos o geomagnéticos de bajo consumo para espacios subterráneos o estrechos, comunicados mediante MQTT/LoRaWAN. Los modelos predictivos de demanda entrenados con ocupación histórica, eventos locales y patrones climáticos permitirían sugerir precios dinámicos.

Integración de operadores comerciales. Adaptar el motor de reservas y el sistema administrativo a la complejidad operativa de las playas grandes: gestión de inventario mixto (rotación por hora frente a titulares de abonos mensuales), asignación de pisos y sectores, informes de auditoría financiera e integración directa con los sistemas electromecánicos de barreras existentes.

Expansión regional e internacional. Desplegar la plataforma en ciudades latinoamericanas con congestión urbana comparable o mayor (São Paulo, Ciudad de México, Bogotá, Santiago y Lima) y en ciudades estadounidenses de alta demanda (Miami, Los Ángeles, Nueva York y Chicago), con soporte completo de internacionalización (español, inglés y portugués), procesamiento de pagos multimoneda mediante Stripe y PayPal, y cumplimiento normativo del RGPD y de la CCPA/CPRA de California.

Estas líneas consolidan la trayectoria de ParkLink desde una plataforma colaborativa de reservas de estacionamiento hacia una infraestructura integral e inteligente de movilidad urbana.

Referencias

[1] Ferguson, T.S.: Who solved the secretary problem? Statistical Science **4**(3), 282–289 (1989)

[2] MacQueen, J., Miller, R.G.: Optimal persistence policies. Operations Research **8**(3), 362–380 (1960)

[3] Christian, B., Griffiths, T.: Algorithms to Live By: The Computer Science of Human Decisions. Macmillan, New York (2016)

[4] Tamaki, M.: A secretary problem with uncertain employment. Journal of Applied Probability **25**(2), 360–366 (1988)

[5] Blinkay: Estacionamiento medido en Argentina. `https://www.blinkay.com` (2026)

[6] SpotHero: Find and reserve parking online. `https://spothero.com` (2026)

[7] EasyPark Group: Smart parking solutions. `https://easyparkgroup.com` (2026)

[8] Waze: Free community-based GPS navigation app. `https://www.waze.com` (2026)

[9] Google LLC: Google Maps. `https://maps.google.com` (2026)

[10] Mercado Libre: Plataforma de compraventa online. `https://www.mercadolibre.com.ar` (2026)

[11] SuperAuto: ¿Cuántos automóviles ingresan diariamente a Capital Federal? `https://superauto.com.ar` (2024)

[12] Meta Open Source: React Native. `https://reactnative.dev` (2026)

[13] The PostgreSQL Global Development Group: PostgreSQL 16. https://www.postgresql.org (2026)

[14] PostGIS Project: PostGIS — Spatial and Geographic Objects for PostgreSQL. https://postgis.net (2026)

[15] Broadcom: RabbitMQ — Open Source Message Broker. https://www.rabbitmq.com (2026)

[16] Google LLC: Firebase Cloud Messaging. https://firebase.google.com (2026)